

CRUCIBLE — recording script

Segments N1 to N5. One voice note per segment.

Before the first take: record 60 seconds of room tone. Sit still, engine off, say nothing. It is the single most useful thing on the card and it cannot be recreated later.

1. One take per segment. They cut together, so a fluffed line costs one segment and not the read.
2. Slate each take out loud — “N3, take one” — then wait two seconds before the first word.
3. Two seconds of silence at the end too, before you stop recording.
4. Read at your own pace. **The timings below are estimates, not targets.** The locked script says the same thing: the narration’s real length sets the cue list, not the other way round.
5. Grey boxes are what appears on screen. **Do not read them aloud.**
6. If you stumble, pause, breathe, and start that paragraph again. Do not start the segment over.

Segment	What it is	Words	Estimate at 150 wpm	Script target
N1	0:00–0:12 · cold open, no title card	11	0:04	0:00 to 0:12
N2	0:12–0:25 · establish that it really moves money	33	0:13	0:12 to 0:25
N3	0:25–0:50 · the friction	58	0:23	0:25 to 0:50
N4	0:50–1:35 · architecture · diagram on screen throughou	261	1:44	0:50 to 1:35
N5	1:35–1:43 · the honesty beat	74	0:29	1:35 to 1:43
Total		437	2:54	-

N1 of 5

Before you read: say “N1, take one” out loud, then wait two seconds.

0:00–0:12 • cold open, no title card • 11 words • about 0:04 at 150 wpm • script says 0:00 to 0:12

ON SCREEN (NOT READ ALOUD)

refund agent UI. Type: *I got a cracked mug in order 4471, it was

This is a customer service agent with permission to move money.

N2 of 5

Before you read: say “N2, take one” out loud, then wait two seconds.

0:12–0:25 • establish that it really moves money • 33 words • about 0:13 at 150 wpm • script says 0:12 to 0:25

ON SCREEN (NOT READ ALOUD)

```
– type exactly this. The old amount_cents version errors live  
sqlite3 ledger.db "select order_id, amount_minor, ts from refunds order by ts desc  
limit 1"
```

That's not a mock response. The ledger moved.

Now the same agent on a nine hundred dollar claim.

— *let it escalate on screen]] It's a good agent. It follows its polic* —

Which is the problem. Its policy is a paragraph of English,
and English is attackable.

N3 of 5

Before you read: say “N3, take one” out loud, then wait two seconds.

0:25–0:50 • the friction • 58 words • about 0:23 at 150 wpm • script says 0:25 to 0:50

ON SCREEN (NOT READ ALOUD)

one slide — *find the breach · patch it · prove the patch didn't

Before you deploy an agent with real permissions, someone has to find out what it does under pressure.

Today that's a person writing prompts by hand until they get bored. There's no regression suite, so last week's fix is untested this week.

CRUCIBLE automates that loop. And more importantly, it refuses to ship a fix it can't prove.

N4 of 5

Before you read: say “N4, take one” out loud, then wait two seconds.

0:50–1:35 · architecture · diagram on screen throughout · 261 words · about 1:44 at 150 wpm · script says 0:50 to 1:35

A red team model writes six attacks a round against the target.

The attack corpus was authored, then sealed and committed before the first patch existed. That commitment is public and timestamped, and the identity that writes patches cannot read the sealed set.

The tripwire is pure code. No model. It rules from the actual tool call trace, not from anything the agent said, against eleven clauses.

The coroner writes the autopsy and structurally cannot propose a fix. There is no fix field in its schema, its findings sit in a subtree the armorer's input adapter cannot address, and its service account has no write access to the policy bucket. That's an IAM policy, not a prompt instruction.

The armorer gets structured fields only, and patches in three verbs. Deny. Constrain argument. Require approval. There is no allow verb, so no sequence of patches can widen what the agent is permitted to do. The predicates reference trace facts and never match strings. That constraint is the whole design.

The compiled policy runs as a plugin callback, before the agent's own callbacks, and a non-None return skips execution. So the policy cannot be argued with by the agent it governs.

Then the regression warden, also pure code. Twenty six benign fixtures, fourteen of them near misses, and the patch has to leave all twenty six passing.

The gate promotes only if attack success falls AND benign is exactly one hundred percent.

Above this line is model generated and untrusted. Below it is deterministic code.

No model ever decides whether a breach happened.

N5 of 5

Before you read: say “N5, take one” out loud, then wait two seconds.

1:35–1:43 • the honesty beat • 74 words • about 0:29 at 150 wpm • script says 1:35 to 1:43

Everything from here is replayed from stored evidence bundles, recorded offline.

Vertex runs on dynamic shared quota, so a multi-minute live loop on camera is a risk I'm not taking.

Every bundle carries five locks. The gate rule, the frozen target agent, the capability manifest, the Objective Set that defines what counts as a breach, and the corpus. All committed before the first measurement, and the replay tool needs no credentials to check them.